

Pillar ② — Infrastructure: Scan → Blueprint

July 17, 2026

CONTENTS

name: infrastructure-scan-blueprint description: > Pillar ② of the AI Infrastructure Protocol — the heart. Runs G0 SCAN → G1 BLUEPRINT

Pillar ② — Infrastructure: Scan → Blueprint

Where the evidence lives

G0 — SCAN

What gets captured — per process, non-negotiable

How to run the scan

Output — company-scan.json

Gate 0 green — criteria

G1 — BLUEPRINT

The disposition decision — default to automate

Name the artifact — every augment/automate item

Rank the backlog — impact x effort, reproducibly

The AI-involvement score — the number the product is judged on

Output — blueprint.json

Gate 1 green — criteria

Scan → Blueprint pipeline (full)

Handing the backlog to G2

name: infrastructure-scan-blueprint description: > Pillar ② of the AI Infrastructure Protocol — the heart. Runs G0 SCAN → G1 BLUEPRINT: a structured audit of a company's entire operation, then the design of its target AI operating model with maximum AI involvement. Use when opening an AI-infrastructure engagement, scanning a company's departments/processes, producing company-scan.json, deciding keep-human / augment-AI / fully-automate per process, or producing blueprint.json and the ranked build backlog handed to G2. Reads the spine at ../../PROTOCOL.md; this skill governs G0 and G1 only.

Pillar ② — Infrastructure: Scan → Blueprint

This is the heart of the protocol. Everything downstream — the apps built by the four-gate engine, the agents from the agent factory, the security gate, the investor package — is scoped by what happens here. Get the scan shallow and the whole company gets a shallow AI operating system. Get the blueprint timid and you sold automation and shipped a chatbot.

Two phases, two gates:



Two non-negotiable objective functions govern every decision in this pillar:

1. **Measure before you design.** No disposition is allowed on a process you did not quantify. Volume, hours, and pain are the fuel; without them the backlog ranking is a guess wearing a number.
2. **Maximize AI involvement.** The default disposition is `fully-automate`. `augment-ai` and `keep-human` are exceptions that must be *justified against the default*, not chosen because they feel safer. “We’d prefer a person” is not a justification. A regulatory mandate is.

Everything here is an instruction, not a suggestion. When this skill and your instinct disagree, this skill wins; if it is genuinely wrong for a client, log it in `.protocol/journal.jsonl` and keep moving.

Where the evidence lives

.protocol/

├ company-scan.json

├ blueprint.json

├ gates/

├ gate-0.json

├ gate-1.json

├ evidence/

├ scan-coverage.txt

├ scan-inventory.txt

├ blueprint-ranking.txt

└ journal.jsonl

← G0 output (validated against templates/company-scan.schema.json)

← G1 output (validated against templates/blueprint.schema.json)

← SCAN gate state

← BLUEPRINT gate state

← which departments/processes were captured, and gaps

← agent-browser/scrape tool-inventory output

← the backlog ranking math, reproducible

← every intervention and note, append-only

Gate discipline matches the app engine: a gate is a check that writes `gate-N.json`; G1 refuses to open while G0 is red; a green gate goes **stale** the moment its input file changes underneath it (edit `company-scan.json` after Gate 0 green and Gate 0 is stale until re-checked). Never hand-edit a gate file.

G0 — SCAN

Read the whole company. Every process, every tool, every data source, every handoff, every hour, every pain. Output the as-is operating map.

This is a **structured audit, not a survey**. A survey asks “how’s your ops?” and gets adjectives. An audit walks each department, enumerates each process, and pins a number to every one. The schema (`templates/company-scan.schema.json`) is the contract for what “captured” means.

WHAT GETS CAPTURED — PER PROCESS, NON-NEGOTIABLE

For **every** process in **every** department, capture all eight:

4 of 9

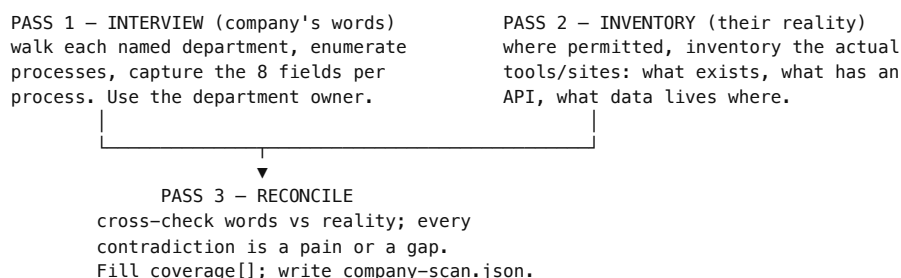
CONFIDENTIAL

Field	What it is	Why it matters downstream
steps	the ordered actions, in the company's words, marking decision points	the automation surface — vague steps cap the blueprint
tools	every system it touches; API? inspected live? cost?	feasibility + G4 unit economics
data	every object read/written + sensitivity	routes the artifact through Pillar ⑤ Security
volume	count × unit × period (numeric)	impact weighting in the backlog
hours	human hours/period × people × loaded €/hr	"cost taken out" number for G4
pains	specific errors/delays/rework with evidence	what the AI is designed <i>against</i>
handoffs	who passes work to whom, via what, wait time	where automation pays off most
trigger	what starts it	agent/automation entry point at G2

Rule: **"unknown" is not a value.** A process with unquantified volume or hours is not scanned — it is a gap, and gaps go in `coverage.gaps[]` with a reason, never silently omitted. You cannot rank a backlog you did not measure.

HOW TO RUN THE SCAN

Run three passes and merge. Do not try to capture a company in one linear interview — you will miss the processes nobody thinks to mention.



Pass 1 — Interview. Start from `company.namedDepartments` (captured in kickoff). For each department, sit with the owner and enumerate processes. For each process, drive the eight fields to a number. Anchor hours with a concrete recent instance (“walk me through the last invoice you approved — what did you touch, how long?”) not an average they’ll round.

Pass 2 — Inventory (the differentiator). The company’s account of its tools is always incomplete. Where the client has **recorded permission** (schema: `scan.permissions`), inventory their live stack directly — this is what makes the scan an audit, not a form:

- **agent-browser** — log into their web tools and map what actually exists: which CRM pipelines, which spreadsheet tabs feed which reports, which admin panels, how many open tickets, what integrations are already wired. Reuse the bundled `skills/agents/agent-browser` skill; drive it headless, capture to `.protocol/evidence/scan-inventory.txt`.
- **scrape** — pull their public site, docs, help center, pricing to understand the customer-facing surface and the content operation behind it. Bundled at `skills/agents/scrape`.
- **composio-cli** — probe which of their SaaS tools expose APIs/webhooks (sets `tools[].hasApi`, which directly gates full-automation feasibility at G1). Bundled at `skills/agents/composio-cli`.

Never automate or scrape a tool that is not listed under `scan.permissions.toolsInventoried` with `consentRecordedBy` set. Inventorying without recorded consent is a Gate 0 red — Security (Pillar ⑤) is a standard the scan is held to too.

Pass 3 — Reconcile. Put the interview against the inventory. Every mismatch is signal: a tool nobody mentioned is a shadow process to scan; a process with no tool is either manual drudgery (high automation value) or under-captured. Fill `coverage`, list every `gap` honestly, write `company-scan.json`.

For a multi-department company, **dispatch the passes in parallel** — one agent per department using `skills/agents/dispatching-parallel-agents` — then merge the per-department `departments[]` into one scan. Faster, and it keeps each department’s detail from being flattened.

OUTPUT — COMPANY-SCAN.JSON

Conforms to `templates/company-scan.schema.json`. Shape:

```
company      { name, industry, headcount, stack[], namedDepartments[] }
scan         { scannedAt, operator, method[], permissions{} }
departments[] → { name, owner, processes[] → {
    id, name, trigger,
    steps[] {order, action, actor, system, decision},
    tools[] {name, category, hasApi, inspected, monthlyCost},
    data[] {name, sensitivity, flow, locatedIn},
    volume {count, unit, period},
    hours {perPeriodHours, period, peopleInvolved, loadedHourlyCost},
    pains[] {description, type, severity, evidence},
    handoffs[] {from, to, via, waitHours}
  }}
coverage     { departmentsScanned[], processesScanned, gaps[] }
```

GATE 0 GREEN — CRITERIA

Gate 0 is green when ALL hold (each is checkable against the file, not asserted):

1. **Coverage is complete.** Every string in `company.namedDepartments` appears in `coverage.departmentsScanned` and in `departments[].name`. A named department missing from the scan is RED — no cherry-picking the easy departments.
2. **Every process is quantified.** For every process: `steps` non-empty, `tools` non-empty, `data` non-empty, `volume` fully numeric, `hours` fully numeric, `pains` non-empty. Any `unknown / n/a` on a required numeric is RED.
3. **Consent is recorded** for every tool marked `inspected:true`.
4. **Gaps are declared, not hidden.** `coverage.gaps[]` accounts for every process the company named but that isn't fully captured. An empty gaps array is a positive assertion of full coverage and is checked against the counts.
5. `company-scan.json` validates against the schema.

A missing interview, an un-inventoried tool the client permitted, or a process you skipped is RED — never a silent skip. If a source is genuinely unavailable (client won't grant tool access), that is a declared `gap` with a reason, and the blueprint will mark the affected processes lower-confidence — it is not invisible.

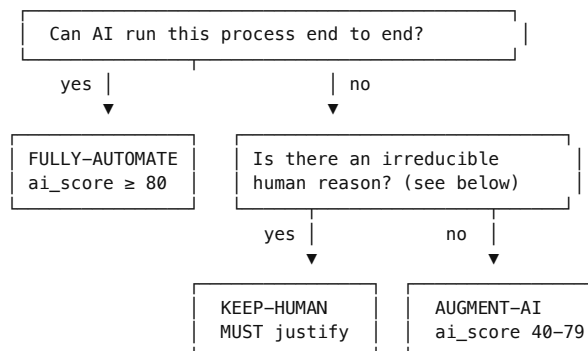
G1 — BLUEPRINT

For every scanned process, decide its future. Name the artifact that delivers it. Rank the build. Output the to-be operating model.

G1 does not open until Gate 0 is green. The blueprint answers exactly the scan — every process id in, a disposition out. The contract is `templates/blueprint.schema.json`.

THE DISPOSITION DECISION — DEFAULT TO AUTOMATE

For each process, choose one:



- **fully-automate** — AI runs it end to end; humans see exceptions only. Target `ai_involvement_score` ≥ 80 . This is the DEFAULT. If you cannot articulate why a process is *not* here, it belongs here.
- **augment-ai** — AI does the heavy lift; a human decides or approves at the judgement steps (the `steps[].decision:true` points from the scan). Typical `ai_involvement_score` 40–79. Choose this when there is a real judgement/decision point that AI drafts but a human owns.
- **keep-human** — no AI in the loop. Allowed ONLY with a `humanOnlyReason` from the fixed set: `regulatory-mandate`, `legal-liability`, `high-stakes-judgement`, `trust-relationship`, `physical-world`, `not-yet-feasible`. A keep-human without one of these is a Gate 1 red. Preference, habit, and “the client is nervous” are not on the list — those are change-management, handled by staging the rollout, not by keeping the process human forever.

The bias is explicit and intentional. When genuinely torn between `augment-ai` and `fully-automate`, pick `fully-automate` and stage the human-approval step as a *removable* checkpoint in the rollout, not a permanent design. When torn between `keep-human` and `augment-ai`, pick `augment-ai`. Maximizing AI involvement is the product.

NAME THE ARTIFACT — EVERY AUGMENT/AUTOMATE ITEM

A disposition without a named artifact is a wish. For every `augment-ai` and `fully-automate` process, name at least one artifact with a `kind`:

kind	is	routes at G2 to
app	a real UI/product (dashboard, portal, tool)	Pillar ① → <code>3day-protocol/</code> four-gate engine
agent	an autonomous LLM worker that acts	Pillar ③ agent factory
automation	a triggered no-/low-code flow (webhook→action)	Pillar ③ agent factory
workflow	a multi-step orchestration across tools	Pillar ③ agent factory
integration	glue wiring two systems (API/webhook)	Pillar ③ agent factory
knowledge-base	a curated, queryable corpus (RAG)	Pillar ③ agent factory

Each artifact records: `delivers` (the outcome in one line), `replacesSteps` (which scan step orders it absorbs — makes before/after auditable), `buildsOnSkill[]` (the bundled skills the operator will compose — reuse before build), and `dataSensitivity` (carried from the scan, drives the G3 security tier).

The choice between `app` and the agent-factory kinds is the same fork the app engine draws: if the deliverable is something a person opens and uses, it's an `app`; if it runs without a person watching, it's an `agent / automation / workflow`.

RANK THE BACKLOG — IMPACT × EFFORT, REPRODUCIBLY

Every named artifact becomes one `backlog[]` item. Rank is not a vibe:

```

impact.score ← derived from scan: volume × hours × max(pain.severity)
               anchored by impact.hoursSavedPerMonth (traces to scanned hours)
effort.score  ← buildDays + blockers (no API? needs OAuth? egress-blocked?)
priorityScore = impact.score / effort.score      (higher → build first)
rank          = dense 1..N ordered by priorityScore desc

```

Write the ranking math to `.protocol/evidence/blueprint-ranking.txt` so the order is reproducible, not asserted. Then split the backlog into sprints: `this` (build now), `next`, `later`. G2 will check that every `this` item ships.

THE AI-INVOLVEMENT SCORE — THE NUMBER THE PRODUCT IS JUDGED ON

Every backlog item carries `ai_involvement_score` (0–100): how much of that process AI runs once the artifact ships. 0 = purely human, 100 = zero-human-touch. `fully-automate` targets ≥ 80 ; `augment-ai` sits 40–79. Keep-human processes carry no backlog item and count as 0.

Roll it up in `aiInvolvement`, **weighted by process hours** (so it reflects where the work actually is, not process count):

```
toBePercent =  $\Sigma(\text{process\_hours} \times \text{ai\_involvement\_score}) / \Sigma(\text{process\_hours})$ 
```

`asIsPercent` is the same rollup today (near 0 for most clients). The `asIsPercent → toBePercent` jump is the headline the whole engagement is judged on and the thesis line Pillar ④ leads with.

OUTPUT — `BLUEPRINT.JSON`

Conforms to `templates/blueprint.schema.json`. Shape:

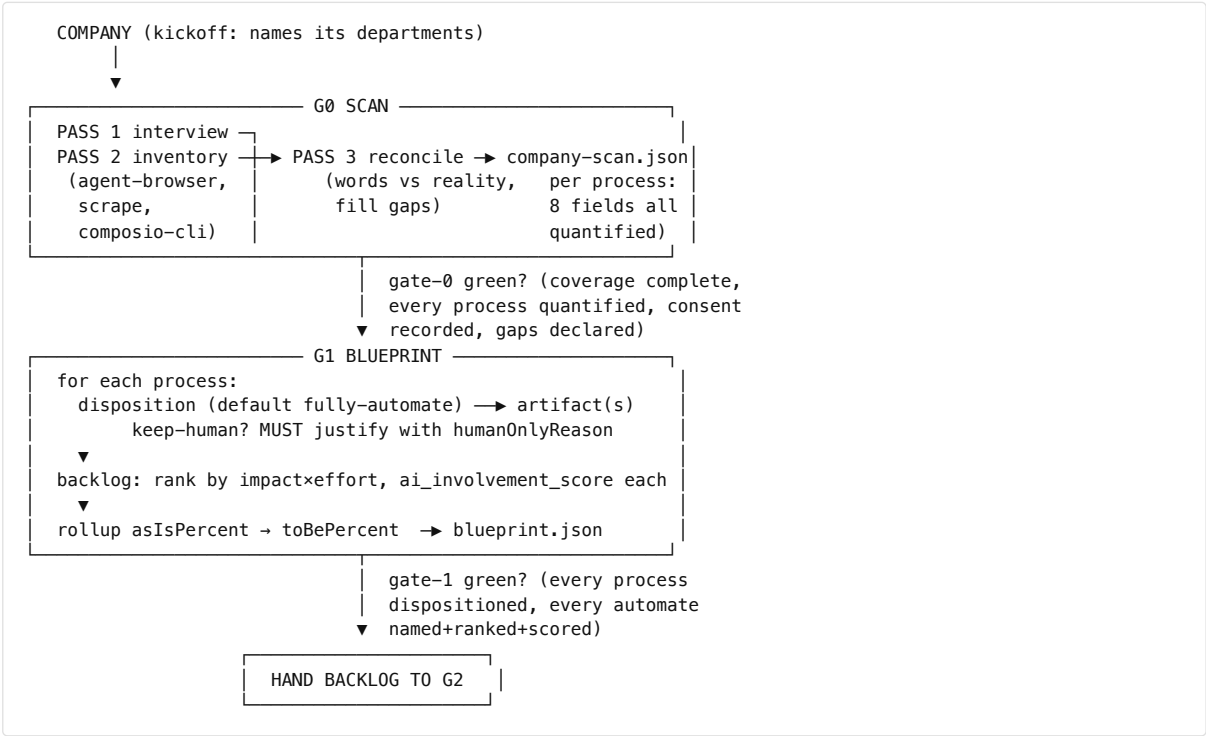
```
company      { name, scanRef }
blueprint    { designedAt, operator, operatingModel }
dispositions[] → { processId, processName, disposition, rationale,
                  humanOnlyReason?, artifacts[] → {
                  id, kind, name, delivers,
                  replacesSteps[], buildsOnSkill[], dataSensitivity
                  }}
backlog[]    → { rank, artifactId, processId, kind,
                  impact{score, hoursSavedPerMonth, eurSavedPerMonth},
                  effort{score, buildDays, blockers[]},
                  priorityScore, ai_involvement_score, sprint, securityTier }
coverage     { processesInScan, dispositionsCount, keepHumanCount,
              augmentCount, automateCount, backlogItems }
aiInvolvement { asIsPercent, toBePercent, method }
```

GATE 1 GREEN — CRITERIA

Gate 1 is green when ALL hold:

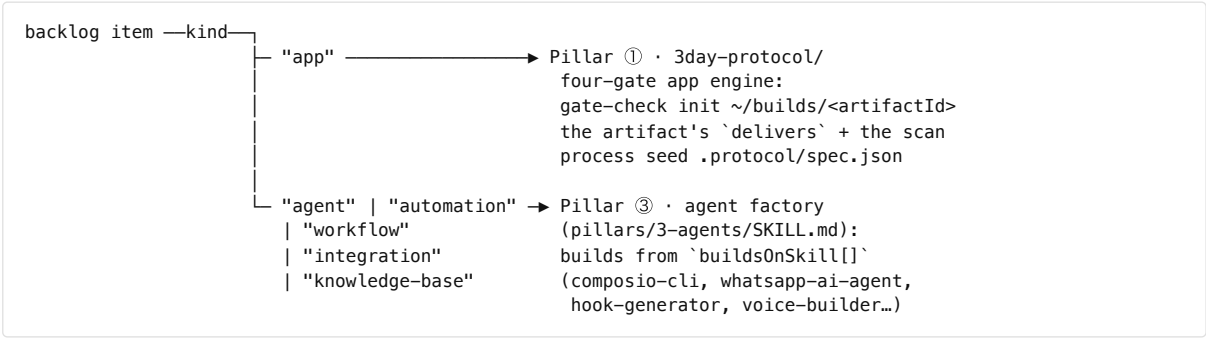
1. **Every scanned process has exactly one disposition.** `coverage.dispositionsCount` equals the scan's process count, and every `processId` in `dispositions[]` matches a scan process id — no gaps, no duplicates, no orphans.
2. **Every keep-human is justified** with a `humanOnlyReason` from the enum. A bare "prefer a person" is RED.
3. **Every augment-ai / fully-automate names ≥ 1 artifact**, and every artifact `id` appears as a `backlog[].artifactId`. A disposition to automate with no artifact is RED.
4. **The backlog is ranked and scored.** `rank` is dense (1..N, unique), `priorityScore` is present and consistent with the rank order, and every item has an `ai_involvement_score`. The ranking math is in evidence.
5. **The AI-involvement rollup is present** with a stated method, and `toBePercent > asIsPercent` (a blueprint that does not raise AI involvement did not do its job).
6. `blueprint.json` validates against the schema.

Scan → Blueprint pipeline (full)



Handing the backlog to G2

G1 green is the trigger. The `backlog[]` (specifically the `sprint:"this"` items, in `rank` order) is the work order for G2 BUILD. Route each item by `kind`:



The handoff contract — what G2 relies on from this pillar:

- 1. **Every `this` -sprint item is buildable as written.** `delivers` is a concrete outcome, `buildsOnSkill[]` names the composition, `effort.blockers[]` lists what must be unblocked first (client OAuth, an API that doesn't exist yet). G2 does not re-scope; if an item can't be built as written, it comes back to G1.
- 2. **Each item carries its `securityTier`** (from `dataSensitivity` + autonomy) so Pillar ⑤ knows at G3 how hard to lean on the finished artifact. `critical` items (regulated data and/or high autonomy) get the full hardening pass; nothing ships G3-unverified regardless.
- 3. **Each built artifact registers in `.protocol/artifacts.json`** at G2 and traces back to its `artifactId` and `processId` here — so the final INFRASTRUCTURE-REPORT can show, per scanned process, the exact artifact that now runs it and its `ai_involvement_score`. Scan process → blueprint disposition → built artifact → security-verified → investor metric: one unbroken chain.

The blueprint is the spine of that chain. If it is honest and ambitious, the company gets a real AI operating system. If it is padded or timid, every gate after it inherits the flaw. Build it deep.